

Go from Data to Strategy ▶

ONLINE MASTER OF SCIENCE IN
BUSINESS ANALYTICSCarnegie Mellon University
Tepper School of Business

Go from Data to Strategy: Tepper School of Business

Using Dropout Regularization in PyTorch Models

by **Adrian Tam** on [April 8, 2023](#) in **Deep Learning with PyTorch**  4 Share Share

Post

Dropout is a simple and powerful regularization technique for neural networks and deep learning models.

In this post, you will discover the Dropout regularization technique and how to apply it to your models in PyTorch models.

After reading this post, you will know:

- How the Dropout regularization technique works
- How to use Dropout on your input layers
- How to use Dropout on your hidden layers
- How to tune the dropout level on your problem

Kick-start your project with my book [Deep Learning with PyTorch](#). It provides **self-study tutorials** with **working code**.

Let's get started.



Using Dropout Regularization in PyTorch Models
Photo by Priscilla Fraire. Some rights reserved.

Overview

This post is divided into six parts; they are

- Dropout Regularization for Neural Networks
- Dropout Regularization in PyTorch

- Using Dropout on the Input Layer
- Using Dropout on the Hidden Layers
- Dropout in Evaluation Mode
- Tips for Using Dropout

Dropout Regularization for Neural Networks

Dropout is a regularization technique for neural network models proposed around 2012 to 2014. It is a layer in the neural network. During training of a neural network model, it will take the output from its previous layer, randomly select some of the neurons and zero them out before passing to the next layer, effectively ignoring them. This means that their contribution to the activation of downstream neurons is temporally removed on the forward pass, and any weight updates are not applied to the neuron on the backward pass.

When the model is used for inference, dropout layer is just to scale all the neurons constantly to compensate the effect of dropping out during training.

Dropout is destructive but surprisingly can improve the model's accuracy. As a neural network learns, neuron weights settle into their context within the network. Weights of neurons are tuned for specific features, providing some specialization. Neighboring neurons come to rely on this specialization, which, if taken too far, can result in a fragile model too specialized for the training data. This reliance on context for a neuron during training is referred to as complex co-adaptations.

You can imagine that if neurons are randomly dropped out of the network during training, other neurons will have to step in and handle the representation required to make predictions for the missing neurons. This is believed to result in multiple independent internal representations being learned by the network.

The effect is that the network becomes less sensitive to the specific weights of neurons. This, in turn, results in a network capable of better generalization and less likely to overfit the training data.

Want to Get Started With Deep Learning with PyTorch?

Take my free email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Download Your FREE Mini-Course

Dropout Regularization in PyTorch

You do not need to randomly select elements from a PyTorch tensor to implement dropout manually. The `nn.Dropout()` layer from PyTorch can be introduced into your model. It is implemented by randomly selecting nodes to be dropped out with a given probability p (e.g., 20%) while in the training loop. In PyTorch, the dropout layer further scales the resulting tensor by a factor of $\frac{1}{1-p}$ so the average tensor value is maintained. Thanks to this scaling, the dropout layer operates at inference will be an identity function (i.e., no effect, simply copy over the input tensor as output tensor). You should make sure to turn the model into inference mode when evaluating the model.

Let's see how to use `nn.Dropout()` in a PyTorch model.

The examples will use the [Sonar dataset](#). This is a binary classification problem that aims to correctly identify rocks and mock-mines from sonar chirp returns. It is a good test dataset for neural networks because all the input values are numerical and have the same scale.

The dataset can be [downloaded from the UCI Machine Learning repository](#). You can place the sonar dataset in your current working directory with the file name `sonar.csv`.

You will evaluate the developed models using scikit-learn with 10-fold cross validation in order to tease out differences in the results better.

There are 60 input values and a single output value. The input values are standardized before being used in the network. The baseline neural network model has two hidden layers, the first with 60 units and the second with 30. Stochastic gradient descent is used to train the model with a relatively low learning rate and momentum.

The full baseline model is listed below:

```
1 import numpy as np
```

```

2 import pandas as pd
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6 from sklearn.preprocessing import LabelEncoder
7 from sklearn.model_selection import StratifiedKFold
8
9 # Read data
10 data = pd.read_csv("sonar.csv", header=None)
11 X = data.iloc[:, 0:60]
12 y = data.iloc[:, 60]
13
14 # Label encode the target from string to integer
15 encoder = LabelEncoder()
16 encoder.fit(y)
17 y = encoder.transform(y)
18
19 # Convert to 2D PyTorch tensors
20 X = torch.tensor(X.values, dtype=torch.float32)
21 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
22
23 # Define PyTorch model
24 class SonarModel(nn.Module):
25     def __init__(self):
26         super().__init__()
27         self.layer1 = nn.Linear(60, 60)
28         self.act1 = nn.ReLU()
29         self.layer2 = nn.Linear(60, 30)
30         self.act2 = nn.ReLU()
31         self.output = nn.Linear(30, 1)
32         self.sigmoid = nn.Sigmoid()
33
34     def forward(self, x):
35         x = self.act1(self.layer1(x))
36         x = self.act2(self.layer2(x))
37         x = self.sigmoid(self.output(x))
38         return x
39
40 # Helper function to train the model and return the validation result
41 def model_train(model, X_train, y_train, X_val, y_val,
42                 n_epochs=300, batch_size=16):
43     loss_fn = nn.BCELoss()
44     optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.8)
45     batch_start = torch.arange(0, len(X_train), batch_size)
46
47     model.train()
48     for epoch in range(n_epochs):
49         for start in batch_start:
50             X_batch = X_train[start:start+batch_size]
51             y_batch = y_train[start:start+batch_size]
52             y_pred = model(X_batch)
53             loss = loss_fn(y_pred, y_batch)
54             optimizer.zero_grad()
55             loss.backward()
56             optimizer.step()
57
58     # evaluate accuracy after training
59     model.eval()
60     y_pred = model(X_val)
61     acc = (y_pred.round() == y_val).float().mean()
62     acc = float(acc)
63     return acc
64
65 # run 10-fold cross validation
66 kfold = StratifiedKFold(n_splits=10, shuffle=True)
67 accuracies = []
68 for train, test in kfold.split(X, y):
69     # create model, train, and get accuracy
70     model = SonarModel()
71     acc = model_train(model, X[train], y[train], X[test], y[test])
72     print("Accuracy: %.2f" % acc)
73     accuracies.append(acc)
74
75 # evaluate the model
76 mean = np.mean(accuracies)
77 std = np.std(accuracies)
78 print("Baseline: %.2f%% (+/- %.2f%%)" % (mean*100, std*100))

```

Running the example generates an estimated classification accuracy of 82%.

```

1 Accuracy: 0.81
2 Accuracy: 0.81
3 Accuracy: 0.76
4 Accuracy: 0.86
5 Accuracy: 0.81
6 Accuracy: 0.90
7 Accuracy: 0.86
8 Accuracy: 0.95
9 Accuracy: 0.65
10 Accuracy: 0.80
11 Baseline: 82.12% (+/- 7.78%)

```

Using Dropout on the Input Layer

Dropout can be applied to input neurons called the visible layer.

In the example below, a new Dropout layer between the input and the first hidden layer was added. The dropout rate is set to 20%, meaning one in five inputs will be randomly excluded from each update cycle.

Continuing from the baseline example above, the code below exercises the same network with input dropout:

```

1 import numpy as np
2 import pandas as pd
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6 from sklearn.preprocessing import LabelEncoder
7 from sklearn.model_selection import StratifiedKFold
8
9 # Read data
10 data = pd.read_csv("sonar.csv", header=None)
11 X = data.iloc[:, 0:60]
12 y = data.iloc[:, 60]
13
14 # Label encode the target from string to integer
15 encoder = LabelEncoder()
16 encoder.fit(y)
17 y = encoder.transform(y)
18
19 # Convert to 2D PyTorch tensors
20 X = torch.tensor(X.values, dtype=torch.float32)
21 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
22
23 # Define PyTorch model, with dropout at input
24 class SonarModel(nn.Module):
25     def __init__(self):
26         super().__init__()
27         self.dropout = nn.Dropout(0.2)
28         self.layer1 = nn.Linear(60, 60)
29         self.act1 = nn.ReLU()
30         self.layer2 = nn.Linear(60, 30)
31         self.act2 = nn.ReLU()
32         self.output = nn.Linear(30, 1)
33         self.sigmoid = nn.Sigmoid()
34
35     def forward(self, x):
36         x = self.dropout(x)
37         x = self.act1(self.layer1(x))
38         x = self.act2(self.layer2(x))
39         x = self.sigmoid(self.output(x))
40         return x
41
42 # Helper function to train the model and return the validation result
43 def model_train(model, X_train, y_train, X_val, y_val,
44                 n_epochs=300, batch_size=16):
45     loss_fn = nn.BCELoss()
46     optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.8)
47     batch_start = torch.arange(0, len(X_train), batch_size)
48
49     model.train()
50     for epoch in range(n_epochs):
51         for start in batch_start:
52             X_batch = X_train[start:start+batch_size]
53             y_batch = y_train[start:start+batch_size]
54             y_pred = model(X_batch)
55             loss = loss_fn(y_pred, y_batch)
56             optimizer.zero_grad()
57             loss.backward()
58             optimizer.step()
59
60     # evaluate accuracy after training
61     model.eval()
62     y_pred = model(X_val)
63     acc = (y_pred.round() == y_val).float().mean()
64     acc = float(acc)
65     return acc
66
67 # run 10-fold cross validation
68 kfold = StratifiedKFold(n_splits=10, shuffle=True)
69 accuracies = []
70 for train, test in kfold.split(X, y):
71     # create model, train, and get accuracy
72     model = SonarModel()
73     acc = model_train(model, X[train], y[train], X[test], y[test])
74     print("Accuracy: %.2f" % acc)
75     accuracies.append(acc)
76
77 # evaluate the model
78 mean = np.mean(accuracies)
79 std = np.std(accuracies)
80 print("Baseline: %.2f%% (+/- %.2f%%)" % (mean*100, std*100))

```

Running the example provides a slight drop in classification accuracy, at least on a single test run.

```

1 Accuracy: 0.62
2 Accuracy: 0.90
3 Accuracy: 0.76

```

```

4 Accuracy: 0.62
5 Accuracy: 0.67
6 Accuracy: 0.86
7 Accuracy: 0.90
8 Accuracy: 0.86
9 Accuracy: 0.90
10 Accuracy: 0.85
11 Baseline: 79.40% (+/- 11.20%)

```

Using Dropout on Hidden Layers

Dropout can be applied to hidden neurons in the body of your network model. This is more common.

In the example below, Dropout is applied between the two hidden layers and between the last hidden layer and the output layer. Again a dropout rate of 20% is used:

```

1 import numpy as np
2 import pandas as pd
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6 from sklearn.preprocessing import LabelEncoder
7 from sklearn.model_selection import StratifiedKFold
8
9 # Read data
10 data = pd.read_csv("sonar.csv", header=None)
11 X = data.iloc[:, 0:60]
12 y = data.iloc[:, 60]
13
14 # Label encode the target from string to integer
15 encoder = LabelEncoder()
16 encoder.fit(y)
17 y = encoder.transform(y)
18
19 # Convert to 2D PyTorch tensors
20 X = torch.tensor(X.values, dtype=torch.float32)
21 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
22
23 # Define PyTorch model, with dropout at hidden layers
24 class SonarModel(nn.Module):
25     def __init__(self):
26         super().__init__()
27         self.layer1 = nn.Linear(60, 60)
28         self.act1 = nn.ReLU()
29         self.dropout1 = nn.Dropout(0.2)
30         self.layer2 = nn.Linear(60, 30)
31         self.act2 = nn.ReLU()
32         self.dropout2 = nn.Dropout(0.2)
33         self.output = nn.Linear(30, 1)
34         self.sigmoid = nn.Sigmoid()
35
36     def forward(self, x):
37         x = self.act1(self.layer1(x))
38         x = self.dropout1(x)
39         x = self.act2(self.layer2(x))
40         x = self.dropout2(x)
41         x = self.sigmoid(self.output(x))
42         return x
43
44 # Helper function to train the model and return the validation result
45 def model_train(model, X_train, y_train, X_val, y_val,
46                 n_epochs=300, batch_size=16):
47     loss_fn = nn.BCELoss()
48     optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.8)
49     batch_start = torch.arange(0, len(X_train), batch_size)
50
51     model.train()
52     for epoch in range(n_epochs):
53         for start in batch_start:
54             X_batch = X_train[start:start+batch_size]
55             y_batch = y_train[start:start+batch_size]
56             y_pred = model(X_batch)
57             loss = loss_fn(y_pred, y_batch)
58             optimizer.zero_grad()
59             loss.backward()
60             optimizer.step()
61
62     # evaluate accuracy after training
63     model.eval()
64     y_pred = model(X_val)
65     acc = (y_pred.round() == y_val).float().mean()
66     acc = float(acc)
67     return acc
68
69 # run 10-fold cross validation
70 kfold = StratifiedKFold(n_splits=10, shuffle=True)
71 accuracies = []
72 for train, test in kfold.split(X, y):
73     # create model, train, and get accuracy
74     model = SonarModel()
75     acc = model_train(model, X[train], y[train], X[test], y[test])
76     print("Accuracy: %.2f" % acc)
77     accuracies.append(acc)

```

```

78
79 # evaluate the model
80 mean = np.mean(accuracies)
81 std = np.std(accuracies)
82 print("Baseline: %.2f%% (+/- %.2f%%)" % (mean*100, std*100))

```

You can see that in this case, adding dropout layer improved the accuracy a bit.

```

1 Accuracy: 0.86
2 Accuracy: 1.00
3 Accuracy: 0.86
4 Accuracy: 0.90
5 Accuracy: 0.90
6 Accuracy: 0.86
7 Accuracy: 0.81
8 Accuracy: 0.81
9 Accuracy: 0.70
10 Accuracy: 0.85
11 Baseline: 85.50% (+/- 7.36%)

```

Dropout in Evaluation Mode

Dropout will randomly reset some of the input to zero. If you wonder what happens after you have finished training, the answer is nothing! The PyTorch dropout layer should run like an identity function when the model is in evaluation mode. That's why you have `model.eval()` before you evaluate the model. This is important because the goal of dropout layer is to make sure the network learn enough clues about the input for the prediction, rather than depend on a rare phenomenon in the data. But on inference, you should provide as much information as possible to the model.

Tips for Using Dropout

The original paper on Dropout provides experimental results on a suite of standard machine learning problems. As a result, they provide a number of useful heuristics to consider when using Dropout in practice.

- Generally, use a small dropout value of 20%-50% of neurons, with 20% providing a good starting point. A probability too low has minimal effect, and a value too high results in under-learning by the network.
- Use a larger network. You are likely to get better performance when Dropout is used on a larger network, giving the model more of an opportunity to learn independent representations.
- Use Dropout on incoming (visible) as well as hidden units. Application of Dropout at each layer of the network has shown good results.
- Use a large learning rate with decay and a large momentum. Increase your learning rate by a factor of 10 to 100 and use a high momentum value of 0.9 or 0.99.
- Constrain the size of network weights. A large learning rate can result in very large network weights. Imposing a constraint on the size of network weights, such as max-norm regularization, with a size of 4 or 5 has been shown to improve results.

Further Readings

Below are resources you can use to learn more about Dropout in neural networks and deep learning models.

Papers

- [Dropout: A Simple Way to Prevent Neural Networks from Overfitting](#)
- [Improving neural networks by preventing co-adaptation of feature detectors](#)

Online materials

- [How does the dropout method work in deep learning?](#) on Quora
- [nn.Dropout](#) from PyTorch documentation

Summary

In this post, you discovered the Dropout regularization technique for deep learning models. You learned:

- What Dropout is and how it works
- How you can use Dropout on your own deep learning models.
- Tips for getting the best results from Dropout on your own models.

Get Started on Deep Learning with PyTorch!

[Learn how to build deep learning models](#)

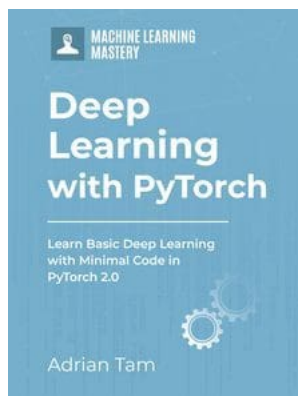
...using the newly released PyTorch 2.0 library

Discover how in my new Ebook:
[Deep Learning with PyTorch](#)

It provides **self-study tutorials** with **hundreds of working code** to turn you from a novice to expert. It equips you with *tensor operation, training, evaluation, hyperparameter optimization*, and much more...

Kick-start your deep learning journey with hands-on exercises

[SEE WHAT'S INSIDE](#)



[Share](#)

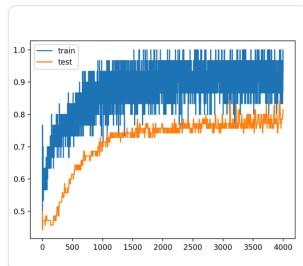
[Share](#)

[Post](#)

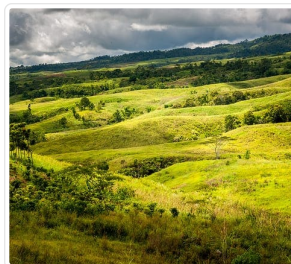
More On This Topic



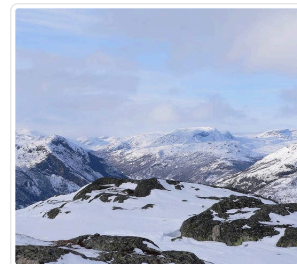
Dropout Regularization in Deep Learning Models with Keras



How to Reduce Overfitting With Dropout...



Use Weight Regularization to Reduce Overfitting of...



Dropout with LSTM Networks for Time Series Forecasting



A Gentle Introduction to Dropout for Regularizing...



Weight Regularization with LSTM Networks for Time...



About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.

[View all posts by Adrian Tam →](#)

[← Loss Functions in PyTorch Models](#)

[Using Learning Rate Schedule in PyTorch Training >](#)

4 Responses to *Using Dropout Regularization in PyTorch Models*

**Ante** February 21, 2023 at 4:44 pm #

REPLY ↩

Thanks, great tutorial.

I am curious how one can use dropout in the INFERENCE stage. Any idea?

The reason for this dropout would be to effectively train only a SINGLE model, but at the same time you wish to have an ENSEMBLE of models (by having different neurons dropped) and use their predictions to estimate the uncertainty of the model predictions.

**Adrian Tam** March 15, 2023 at 5:44 am #

REPLY ↩

It is not usually done but if you insist, PyTorch has the `model.eval()` and `model.train()` to switch between training and inference mode. For many layers they are just the same but drop out layer will toggle between random drop or no drop.

**Ganesh** March 8, 2023 at 6:38 pm #

REPLY ↩

Thanks a great article. Can you please share how KFoldStratified would be run if we use the dataloader and Datasets?

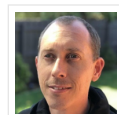
**James Carmichael** March 9, 2023 at 9:39 am #

REPLY ↩

Hi Ganesh....best practices can be found here:

<https://machinelearningmastery.com/repeated-k-fold-cross-validation-with-python/>

Leave a Reply

 Name (required) Email (will not be published) (required)[SUBMIT COMMENT](#)**Welcome!**

I'm *Jason Brownlee* PhD

and I **help developers** get results with **machine learning**.

[Read more](#)

Never miss a tutorial:



Picked for you:



PyTorch Tutorial: How to Develop Deep Learning Models with Python



LSTM for Time Series Prediction in PyTorch



Deep Learning with PyTorch (9-Day Mini-Course)



Calculating Derivatives in PyTorch



Develop Your First Neural Network with PyTorch, Step by Step

Loving the Tutorials?

The [Deep Learning with PyTorch](#) EBook is where you'll find the **Really Good** stuff.

>> SEE WHAT'S INSIDE



Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. [Visit our corporate website](#) to learn more about our mission and team.



[PRIVACY](#) | [DISCLAIMER](#) | [TERMS](#) | [CONTACT](#) | [SITEMAP](#) | [ADVERTISE WITH US](#)

© 2026 Guiding Tech Media All Rights Reserved